

Context-Aware Rate Limiter

Language	Python 3.9+ or Node.js 18+
External state	None. In-memory only. No Redis, no database.

Overview

Most rate limiters count requests and block when a number is exceeded. That is not enough for a real product. A free-tier user hammering an AI endpoint should be stopped much sooner than a paid user reading a list. Your task is to build middleware that makes this distinction.

You will implement a rate limiting layer that reads the caller's tier and the endpoint type from each incoming request, applies the correct rule, and either passes the request through or rejects it with a well-formed error response.

Can refer [rate limiter – smudge.ai blog](#)

Rate Limiting Rules

Enforce exactly the following rules. These are not suggestions - your test cases will verify each cell.

User tier	Endpoint type	Limit	Window
free	AI / heavy	5 requests	60 seconds
free	read / light	30 requests	60 seconds
paid	AI / heavy	30 requests	60 seconds
paid	read / light	120 requests	60 seconds

The **endpoint type** is determined by a route tag you define, not by the URL string. The **user tier** is passed in the request as a header: X-User-Tier with value free or paid.

Requirements

1. Middleware

- Intercept every request before it reaches the route handler.
- Identify the user tier from the X-User-Tier request header. If the header is absent or invalid, treat the caller as free tier.
- Identify the endpoint type from a tag you attach to each route definition (not from parsing the URL).
- Look up the applicable rule and check the caller's current usage against it.
- If the limit is exceeded, return HTTP 429 with the JSON body shown below and do not call the handler.
- If within the limit, increment the counter and pass the request through.

2. In-memory store

- Track usage per unique key: a combination of user identifier, tier, and endpoint type.
- Each key has a request count and a window start timestamp. Reset the count when the window has elapsed.
- Do not use any external process, file, or database. The store lives only in the memory of the running process.

3. 429 response format

```
{
  "error": "rate_limit_exceeded",
  "limit": 5,
  "window_seconds": 60,
  "retry_after_seconds": 34
}
```

`retry_after_seconds` must be the number of seconds remaining in the current window, calculated at the time of rejection. It must never be negative.

4. Routes to expose

Expose at minimum the following four routes for testing:

```
POST /ai/generate      → endpoint type: ai
POST /ai/summarise     → endpoint type: ai
GET  /data/list        → endpoint type: read
GET  /data/export      → endpoint type: read
```

Each route handler should return HTTP 200 with a JSON body of `{ "ok": true }`. The handlers themselves contain no logic - the middleware is the entire focus.

What You Must Not Do

Hard constraints - these will be checked during review

- No Redis, Memcached, or any external caching system.
- No rate-limiting library (express-rate-limit, slowapi, etc.). Implement the logic yourself.
- Do not read the endpoint type from the URL path string. Use a route-level tag or decorator.
- Do not use a database of any kind for state storage.

README Requirements

Your README is evaluated with equal weight to your code. Include the following - in prose, not one-liners.

How to run

Design decisions

Limitations

Evaluation Will Be Based On

Criterion	Points	What we look for
Rule enforcement	30	All four rule combinations produce the correct limit. Limits reset after the window elapses.
429 response format	20	JSON matches the schema. <code>retry_after_seconds</code> is accurate and never negative.
Middleware structure	20	Tier and endpoint type are resolved cleanly. The URL is not parsed to determine endpoint type.
Edge cases	15	Missing header defaults to free tier. Concurrent requests do not corrupt the counter.
README quality	15	Design decisions are explained with reasoning. Limitations name specific failure scenarios.

What we are not evaluating

Framework choice - use FastAPI, Flask, Express, or any other. It does not affect your score.

Response latency or throughput.

Test coverage - welcome but not required.

Instructions

Kindly create a private GitHub repository named: {yourName_Nestack_Submission} (e.g., roy_Nestack_Submission) and add the following contributors for evaluation:

- bishal@nestack.com
- sannidhya@nestack.com
- sanjay@nestack.com

Your submission must include:

- A ZIP file of your complete codebase
- A live deployment link
- Any necessary credentials if your project includes authentication

Additionally, ensure your repository includes a README.md with:

- The deployment link clearly mentioned
- Setup and run instructions if applicable

NOTE: PUBLIC REPOSITORIES ARE DISQUALIFIED FROM THE HIRING PROCESS